DEPARTMENT OF
COMPUTER SCIENCE & ENGINEERING
Discover. Learn. Empower.

Experiment No.: 5

Student Name: Probal Dhali **UID:** 24BAI71013
Branch: AIT-CSE **Section/Group:** B
Semester: 5TH **Date of Performance:**
Subject Name: Competitive Coding – II **Subject Code:** 24CSP-305

Experiment No: 5.1

Aim: To generate all possible subsets of a given set of distinct integers using recursion/backtracking.

Objective: Understand recursion and backtracking for generating the power set of an array.

Problem Statement: Given an integer array nums containing unique elements, return all possible subsets (the power set). The solution set must not contain duplicate subsets.

LeetCode Link: https://leetcode.com/problems/subsets/?utm_source=chatgpt.com

Procedure/Algorithm/Pseudocode:

1. Initialize an empty list to store all subsets.
2. Start from index 0 with an empty current subset.
3. Add the current subset to the result.
4. Iterate through the remaining elements.
5. Include the current element in the subset.
6. Recursively generate subsets from the next index.
7. Remove the element to backtrack.
8. Continue until all possible choices are explored.
9. Return the result.

Code:

```
class Solution {

public:

   vector<vector<int>> subsets(vector<int>& nums) {

      vector<vector<int>> result;

      vector<int> current;

      backtrack(nums, 0, current, result);

      return result;

   }

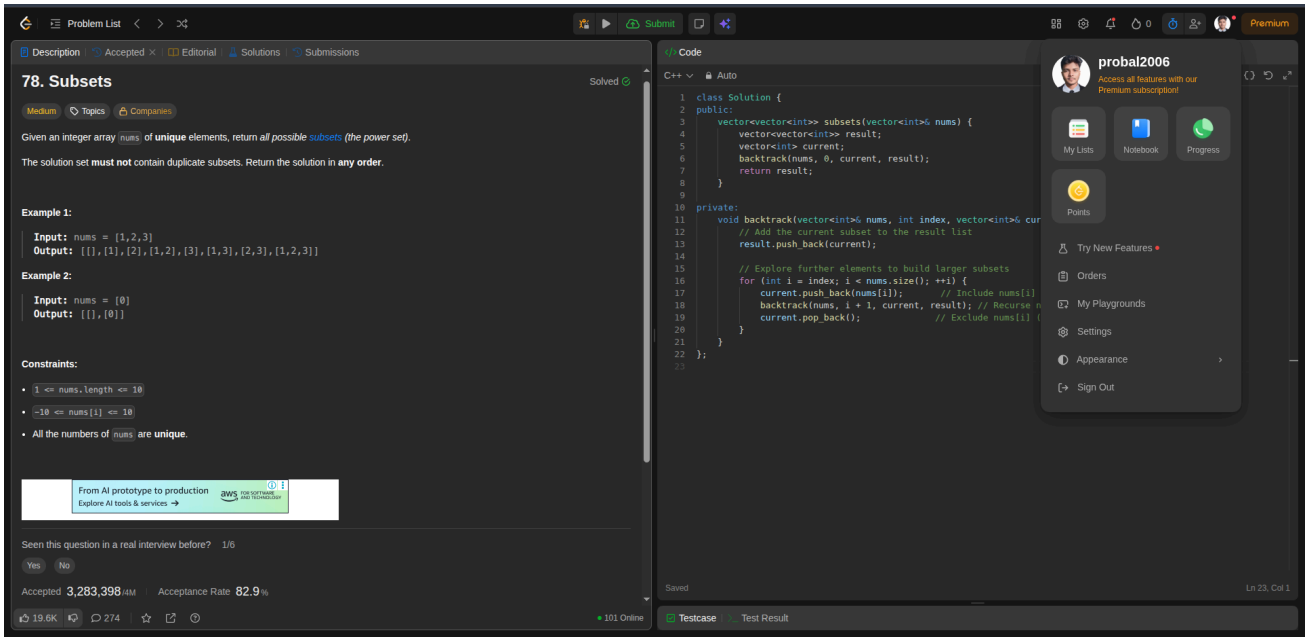
private:

   void backtrack(vector<int>& nums, int index, vector<int>& current, vector<vector<int>>& result) {
```

```
        result.push_back(current);


        for (int i = index; i < nums.size(); ++i) {
            current.push_back(nums[i]);       // Include nums[i]
            backtrack(nums, i + 1, current, result); // Recurse next elements
            current.pop_back();               // Exclude nums[i] (backtrack)
        }
    }
};
```

Output:



Observations/Outcome:

The program successfully generates all possible subsets of the given array. For n elements, the total number of subsets is 2^n.

Time Complexity: $O(n \times 2^n)$

Space Complexity: $O(n)$

Learning outcomes :

- I learned how recursion and backtracking can generate all possible subsets.
- I understood why an array containing n elements has 2^n possible subsets.
- I learned how to include and exclude elements systematically using backtracking.

Experiment No: 5.2

Aim: To find all unique combinations of numbers that add up to a given target using recursion and backtracking.

Objective: To understand how backtracking can be used to explore different combinations while allowing an element to be selected multiple times.

Problem Statement:

Given an array of distinct integers candidates and a target integer target, return a list of all unique combinations of candidates where the chosen numbers sum to target. The same number may be chosen an unlimited number of times.

LeetCode Link: https://leetcode.com/problems/combination-sum/?utm_source=chatgpt.com

Procedure/Algorithm/Pseudocode:

1. Sort the candidate array if required.
2. Initialize an empty result list.
3. Start the backtracking process with index 0 and the target value.
4. If the remaining target becomes 0, store the current combination.
5. If the remaining target becomes negative, stop exploring that path.
6. Iterate through the candidates starting from the current index.
7. Add a candidate to the current combination.
8. Recursively call the function using the same index because an element can be reused.
9. Remove the candidate to backtrack.
10. Continue until all possible combinations are explored.
11. Return the result.

Code:

```
class Solution {

public:

   void findCombinations(int ind, int target, vector<int>& arr, vector<vector<int>>& ans, vector<int>& ds) {

      if (ind == arr.size()) {

         if (target == 0) {

            ans.push_back(ds);

         }

         return;

      }

      if (arr[ind] <= target) {

         ds.push_back(arr[ind]);

         findCombinations(ind, target - arr[ind], arr, ans, ds);

         ds.pop_back(); // Backtrack
```

```
        }

        findCombinations(ind + 1, target, arr, ans, ds);

    }

    vector<vector<int>> combinationSum(vector<int>& candidates, int target) {

        vector<vector<int>> ans;

        vector<int> ds;

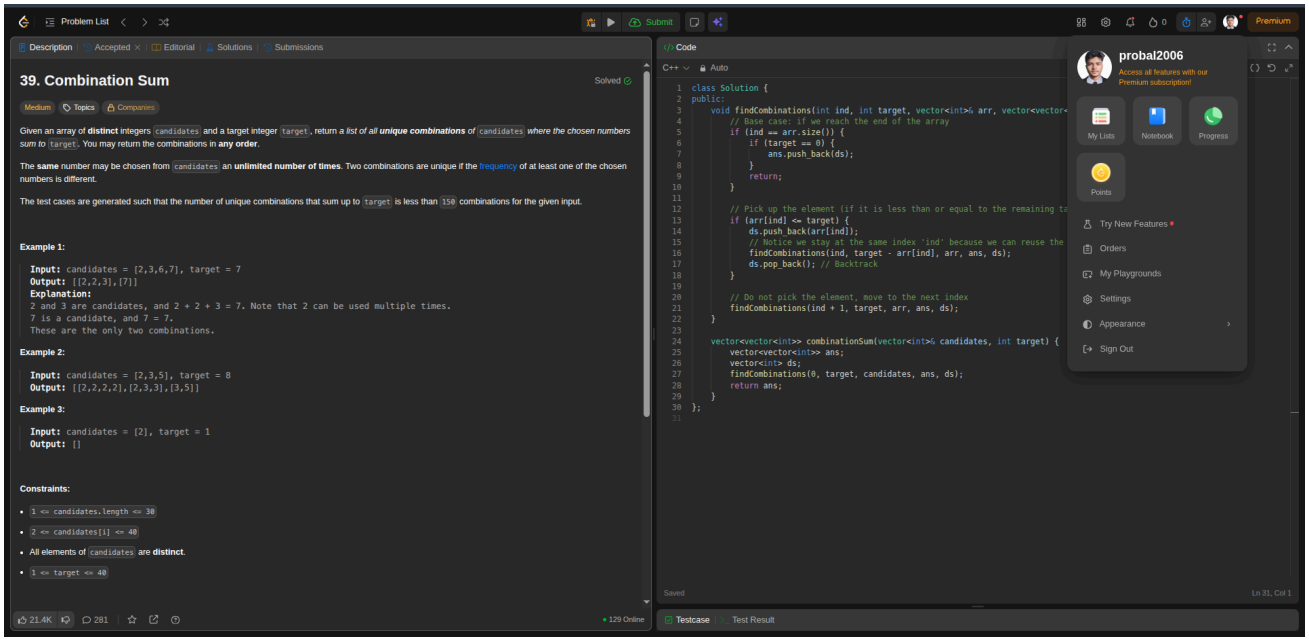
        findCombinations(0, target, candidates, ans, ds);

        return ans;

    }

};
```

Output:



Observations/Outcome:

The program successfully finds all possible combinations whose sum equals the target. Backtracking allows the program to explore different choices and discard invalid combinations when the remaining target becomes negative.

Time Complexity: O(n^(T/M))

Space Complexity: O(T/M)

Learning outcomes:

- I learned how to use recursion and backtracking to solve combination-generation problems.
- I understood how the same element can be reused by recursively calling with the same index.

- I learned how invalid paths can be terminated when the remaining target becomes negative.
- I understood how backtracking removes a selected element and explores another possible choice.